

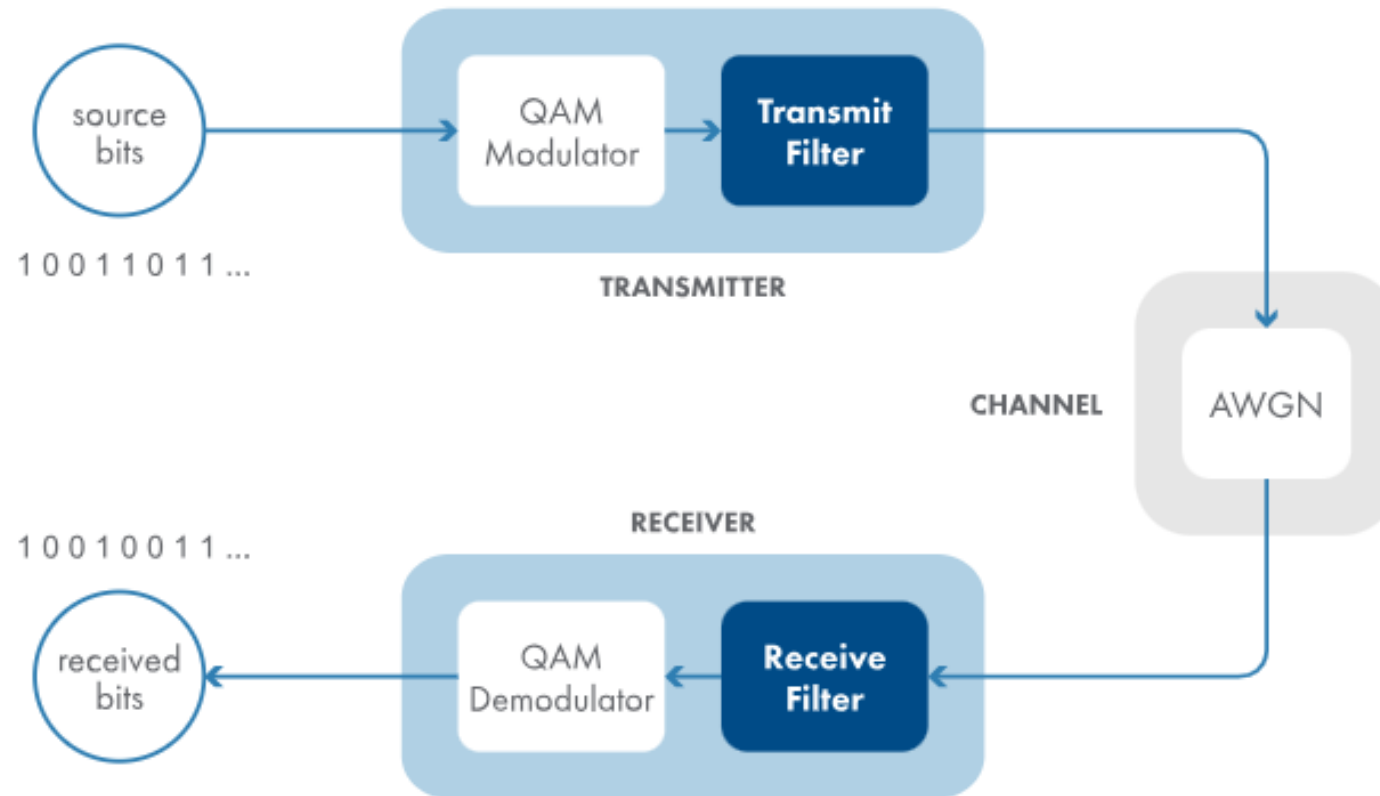
Wireless Communications Using MATLAB

18th Feb 2024

Dr. Saif Ali Jasim Al Grait

Transmit and Receive Filters

In this activity, you'll add square-root raised cosine filters to the transmitter and receiver. The transmit filter takes the 16-QAM signal as input, and its output is sent over the channel. The receive filter takes the channel output as its input, and its output is passed to the QAM demodulator.



You can create a square-root raised cosine transmit filter

Because systems typically use matched filters, it's common to create the transmit and receive filters at the same time.

You can create a square-root raised cosine transmit filter using the following syntax.

```
txFilt = comm.RaisedCosineTransmitFilter
```

A similar syntax creates the matched receive filter.

```
rxFilt = comm.RaisedCosineReceiveFilter
```

These syntaxes set the filter properties to default values. If you leave off the semicolon, you can see some of the properties in the output panel. For example, the property `OutputSamplesPerSymbol` has the default value 8, which means the filter upsamples the signal so it has 8 samples for each QAM symbol.

The raised-cosine filter is a filter frequently used for pulse-shaping in digital modulation due to its ability to minimize inter-symbol interference (ISI)

Generalized Raised-Cosine Filters

Nader Sheikholeslami Alagha and Peter Kabal, *Member, IEEE*

ISI {

Abstract—Data transmission over bandlimited channels requires pulse shaping to eliminate or control intersymbol interference (ISI). Nyquist filters provide ISI-free transmission. Here we introduce a phase compensation technique to design Nyquist filters. Phase compensation can be applied to the square-root of any zero-phase bandlimited Nyquist filter with normalized excess bandwidth less than or equal to one. The resulting phase compensated square-root filter is also a Nyquist filter. In the case of a raised-cosine spectrum, the phase compensator has a simple piecewise-linear form. Such a technique is particularly useful to accommodate two different structures for the receiver, one with a filter matched to the transmitting filter and one without a matched filter.

We also use the phase compensation technique to characterize a more general family of Nyquist filters which subsumes raised-cosine spectra. These generalized raised-cosine filters offer more flexibility in filter design. For instance, the rate of asymptotic decay of the filter impulse response may be increased, or the residual ISI, introduced by truncation of the impulse response, may be minimized. Design examples are provided to illustrate these choices.

Index Terms—Data transmission, intersymbol interference, Nyquist filters, pulse amplitude modulation.

kT_s . In other words, choosing $g(\cdot)$ as a Nyquist pulse avoids intersymbol interference (ISI), and allows sample-by-sample detection at the receiver. In the frequency domain, Nyquist's first criterion is written as

$$\sum_{n=-\infty}^{\infty} G\left(f - \frac{n}{T_s}\right) = T_s \quad (3)$$

where $G(f)$ is known as a Nyquist filter. A particular Nyquist filter with wide practical applications is the *raised-cosine* filter

$$G_{rc}(f) = \begin{cases} T_s, & |f| \leq \frac{1-\alpha}{2T_s} \\ T_s \cos^2\left(\frac{\pi T_s}{2\alpha}\left(|f| - \frac{1-\alpha}{2T_s}\right)\right), & \frac{1-\alpha}{2T_s} \leq |f| \leq \frac{1+\alpha}{2T_s} \\ 0, & |f| > \frac{1+\alpha}{2T_s} \end{cases} \quad (4)$$

“Eye pattern diagrams provide a simple and effective way to measure and visualize the noise immunity of a pulse shaping scheme. Using an eye pattern, one can also assess the effect of errors in the timing phase and the sensitivity to phase jitter”

Fig. 3. Binary eye patterns for a raised-cosine filter and a square-root raised-cosine filter. (a) Raised-cosine filter with $\alpha = 1$. (b) Square-root raised-cosine filter with $\alpha = 1$.

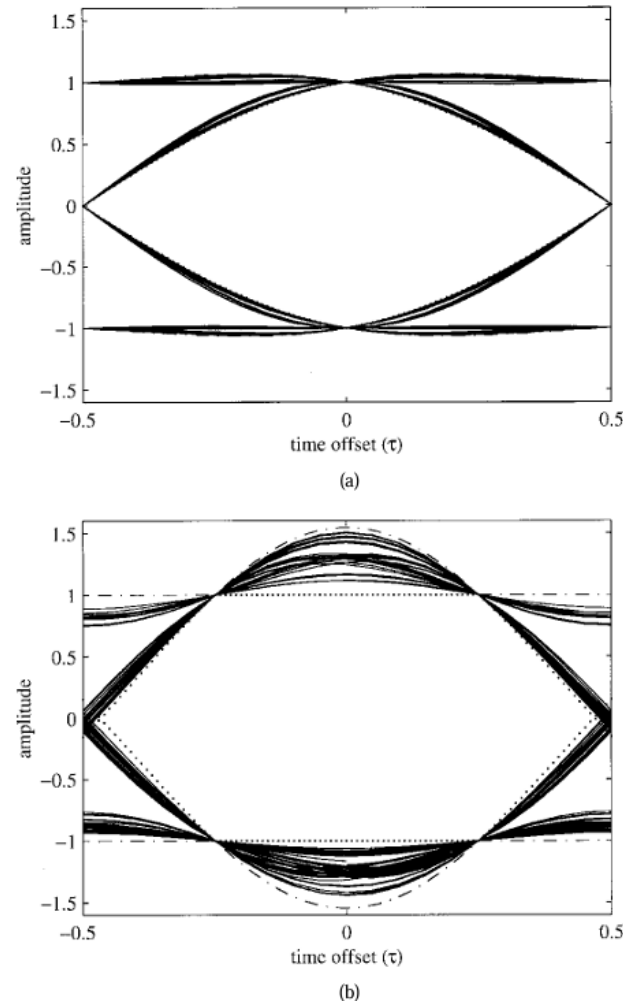
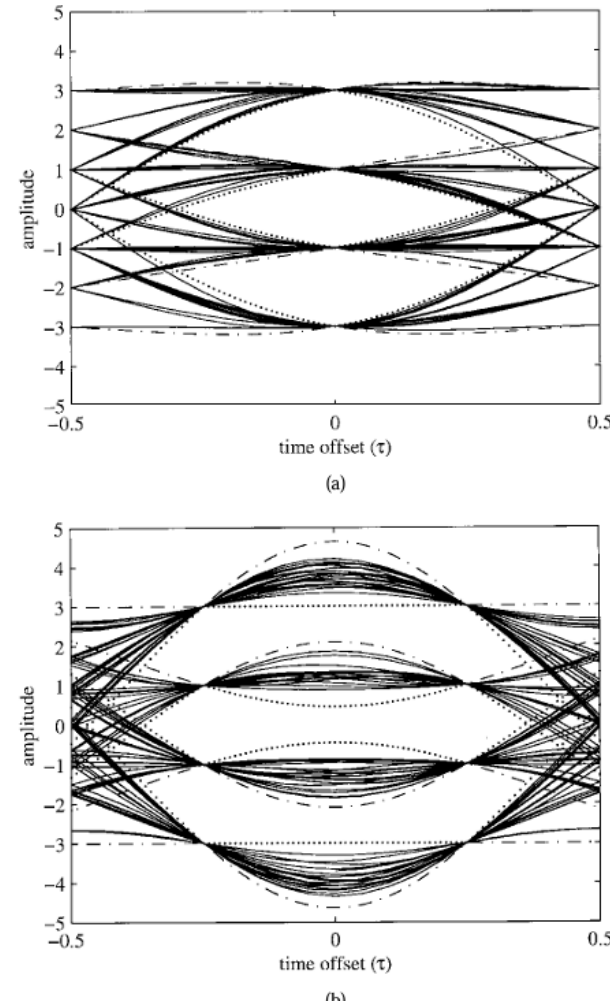


Fig. 4. Eye patterns (4-level PAM) for a raised-cosine filter and a square-root raised-cosine filter. (a) Raised-cosine filter with $\alpha = 1$. (b) Square-root raised-cosine filter with $\alpha = 1$.



You can create a square-root raised cosine transmit filter

Because systems typically use matched filters, it's common to create the transmit and receive filters at the same time.

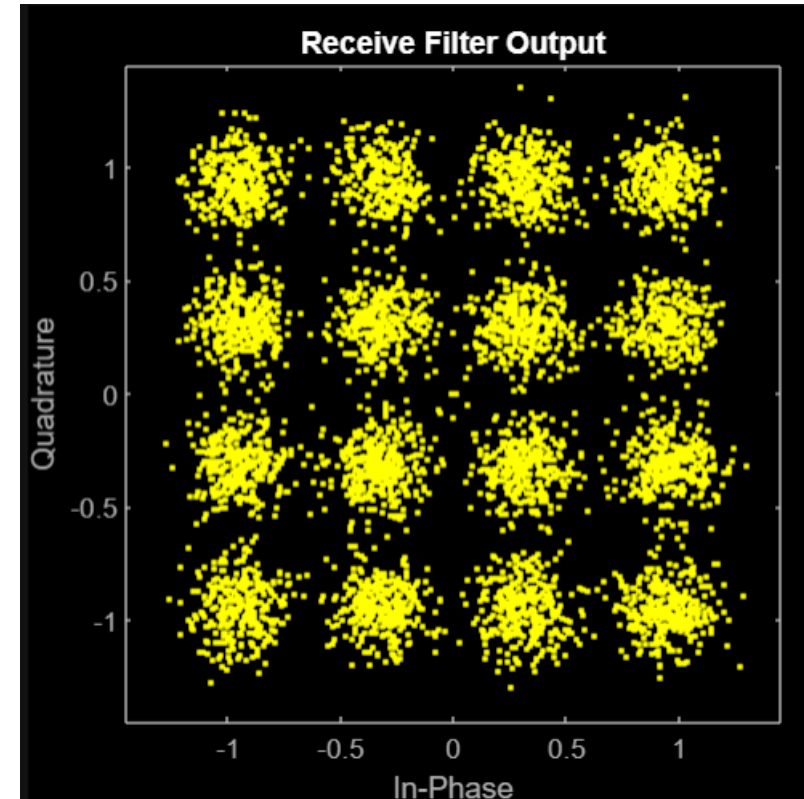
You can create a square-root raised cosine transmit filter using the following syntax.

```
txFilt = comm.RaisedCosineTransmitFilter
```

A similar syntax creates the matched receive filter.

```
rxFilt = comm.RaisedCosineReceiveFilter
```

These syntaxes set the filter properties to default values. If you leave off the semicolon, you can see some of the properties in the output panel. For example, the property `OutputSamplesPerSymbol` has the default value 8, which means the filter upsamples the signal so it has 8 samples for each QAM symbol.



comm.RaisedCosineTransmitFilter

Apply pulse shaping by interpolating signal using raised-cosine FIR filter

Description

The `comm.RaisedCosineTransmitFilter` System object™ applies pulse shaping by interpolating an input signal using a raised cosine finite impulse response (FIR) filter. The FIR filter has $(\text{FilterSpanInSymbols} \times \text{OutputSamplesPerSymbol} + 1)$ tap coefficients.

To apply pulse shaping by interpolating an input signal using a raised cosine FIR filter:

1. Create the `comm.RaisedCosineTransmitFilter` object and set its properties.
2. Call the object with arguments, as if it were a function.

To learn more about how System objects work, see [What Are System Objects?](#)

Creation

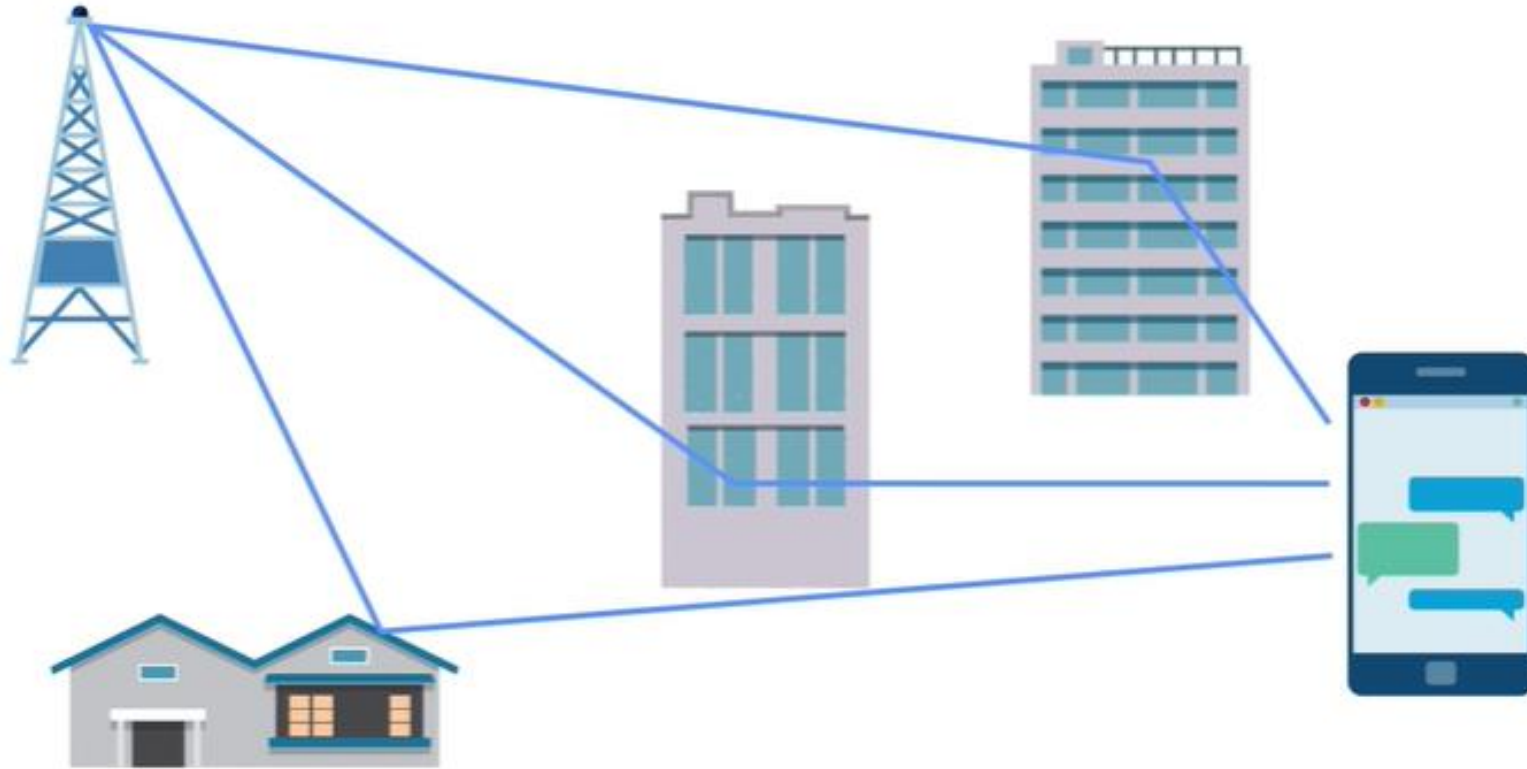
Syntax

```
txfilter = comm.RaisedCosineTransmitFilter
txfilter = comm.RaisedCosineTransmitFilter(Name,Value)
```

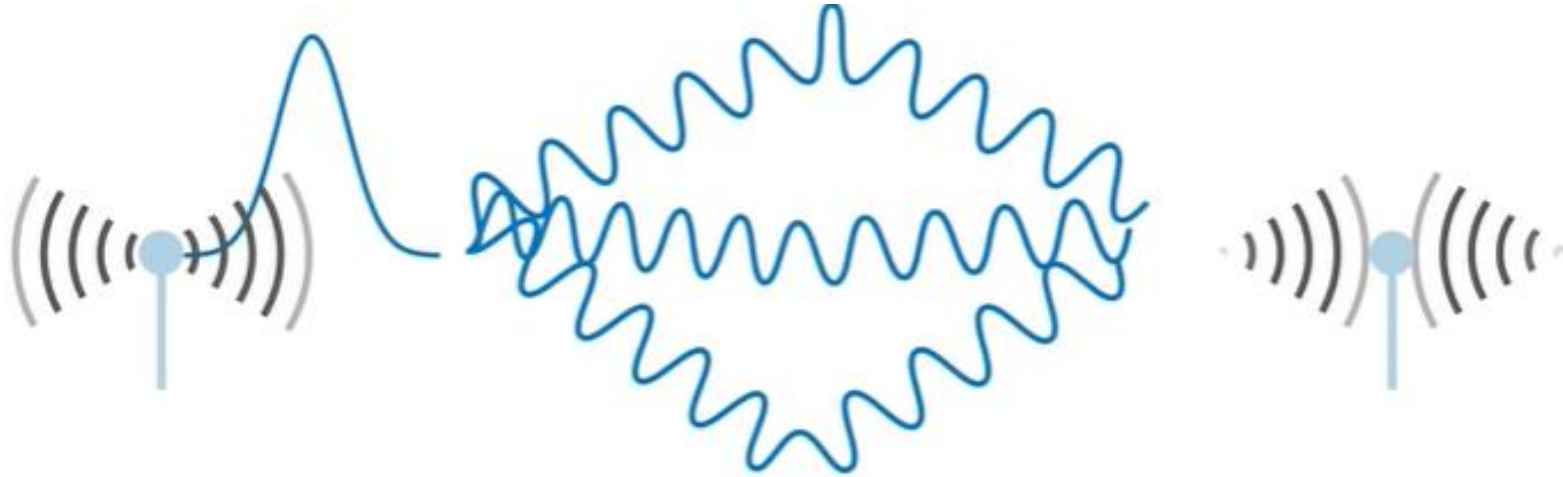
Many modern communications systems like LTE, Wi-Fi, and 5G typically operate where a line of sight doesn't exist between the transmitter and receiver



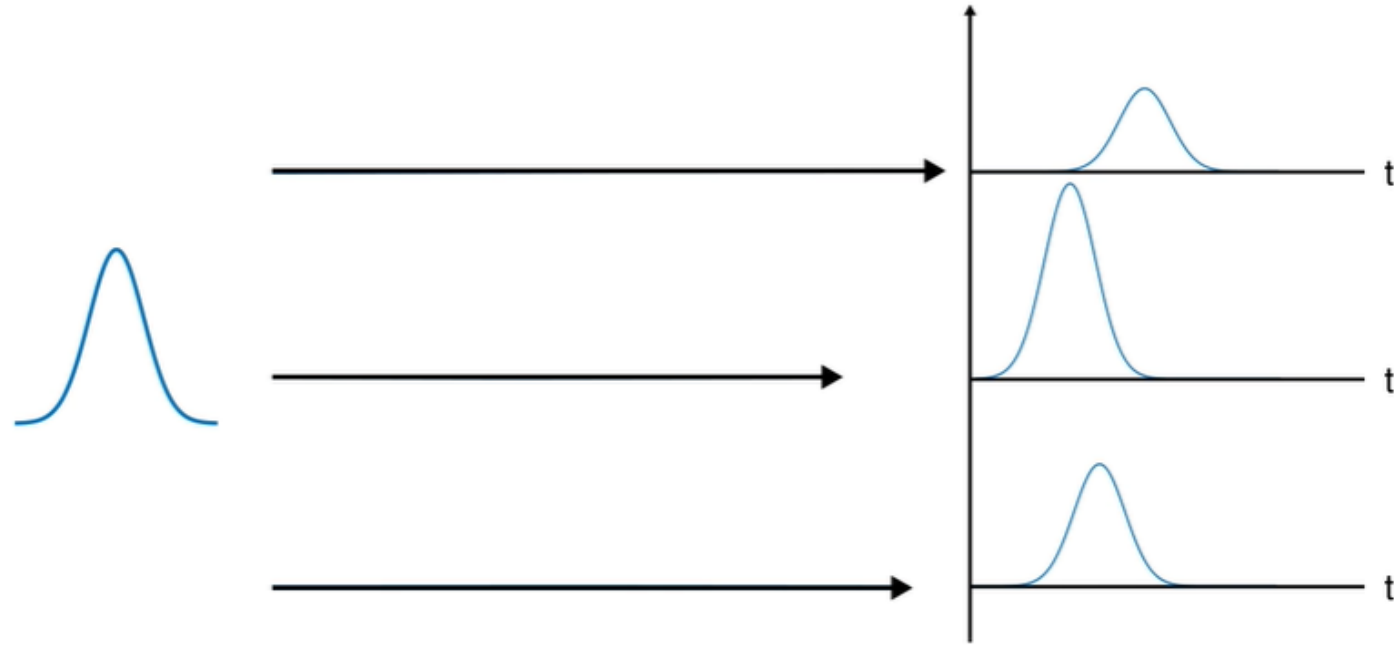
The transmitted signal generally has reflected off many different surfaces before it reaches the receiver



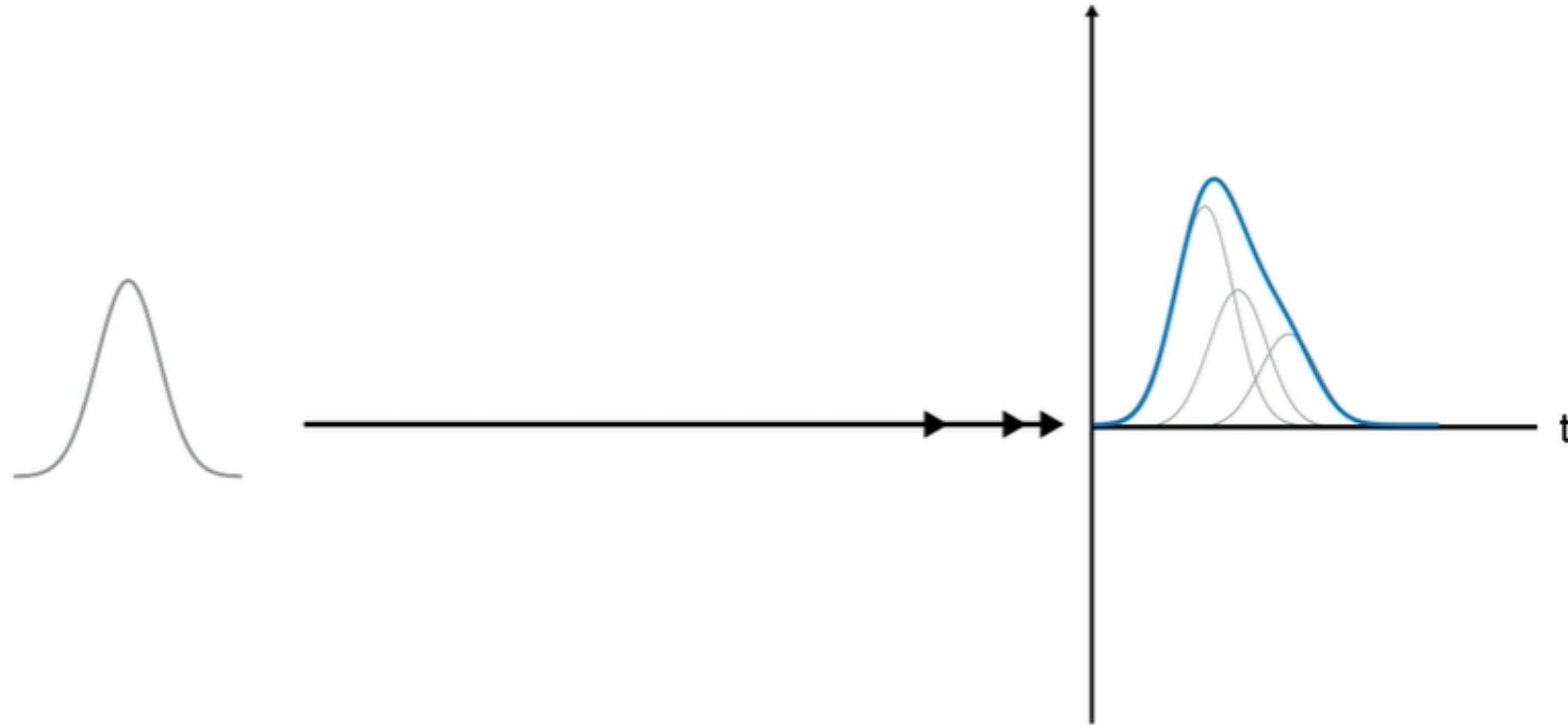
These reflections result in a phenomenon called multipath, where it means the transmitted signal has traveled multiple different path by the time it reaches the receiver



Each path changes the signal while it's on its way, for example it could have a different loss or take longer to arrive

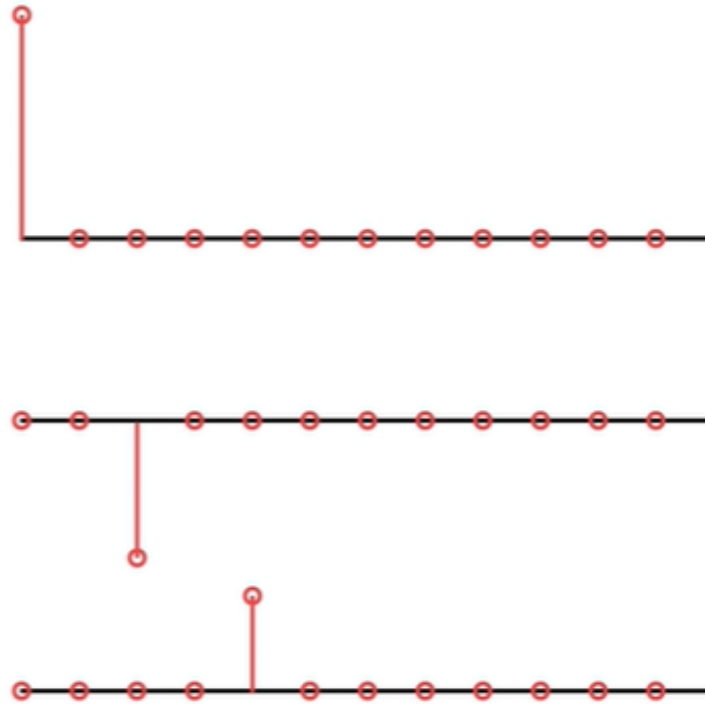


As a result, the receiver gets multiple superimposed copies, and sees time smeared version of the transmitted signal



When modeling multipath in MATLAB it can be represented by impulse response of a digital filter, mathematically it is simply the sum of the impulse response for the various paths

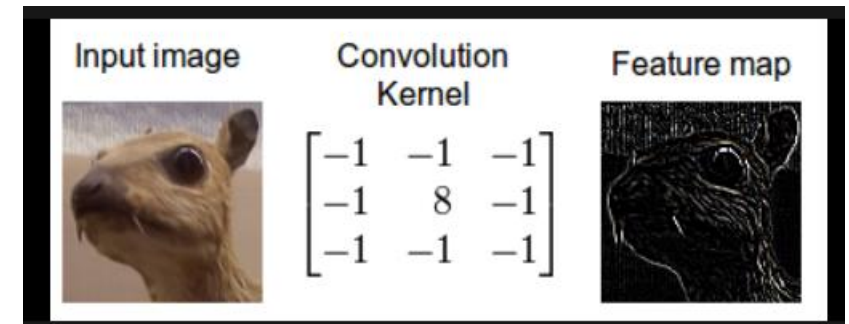
$h[n]$



The impulse response, it is the way a system would respond if an impulse was put into the system so a short sharp input and then how the system responds

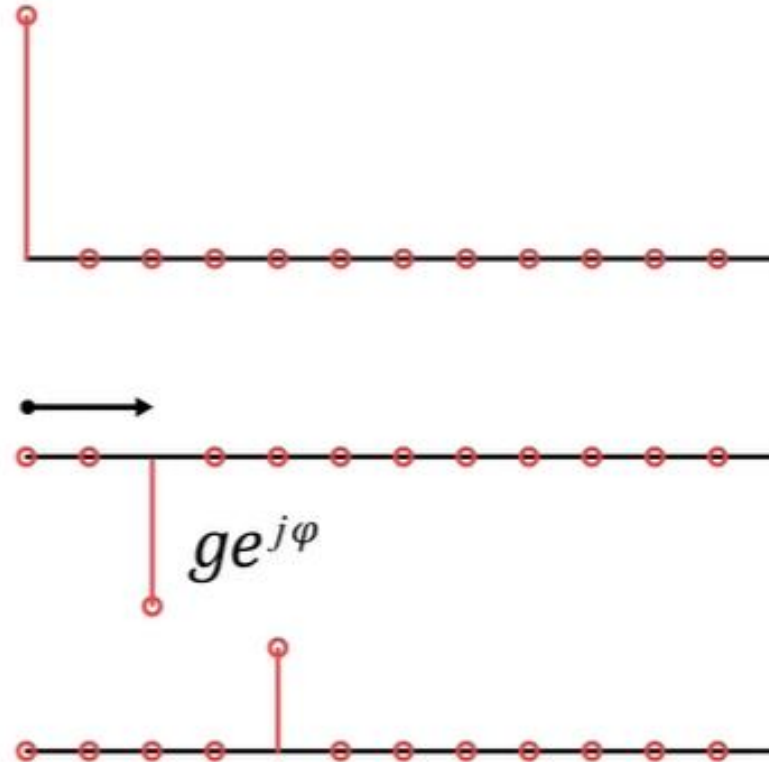


“Convolution is a mathematical way of combining two signals to form a third signal. It is the single most important technique in Digital Signal Processing. Using the strategy of impulse decomposition, systems are described by a signal called the impulse response.” ref {[1](#)}



Including the time delay and gain and phase change for each path

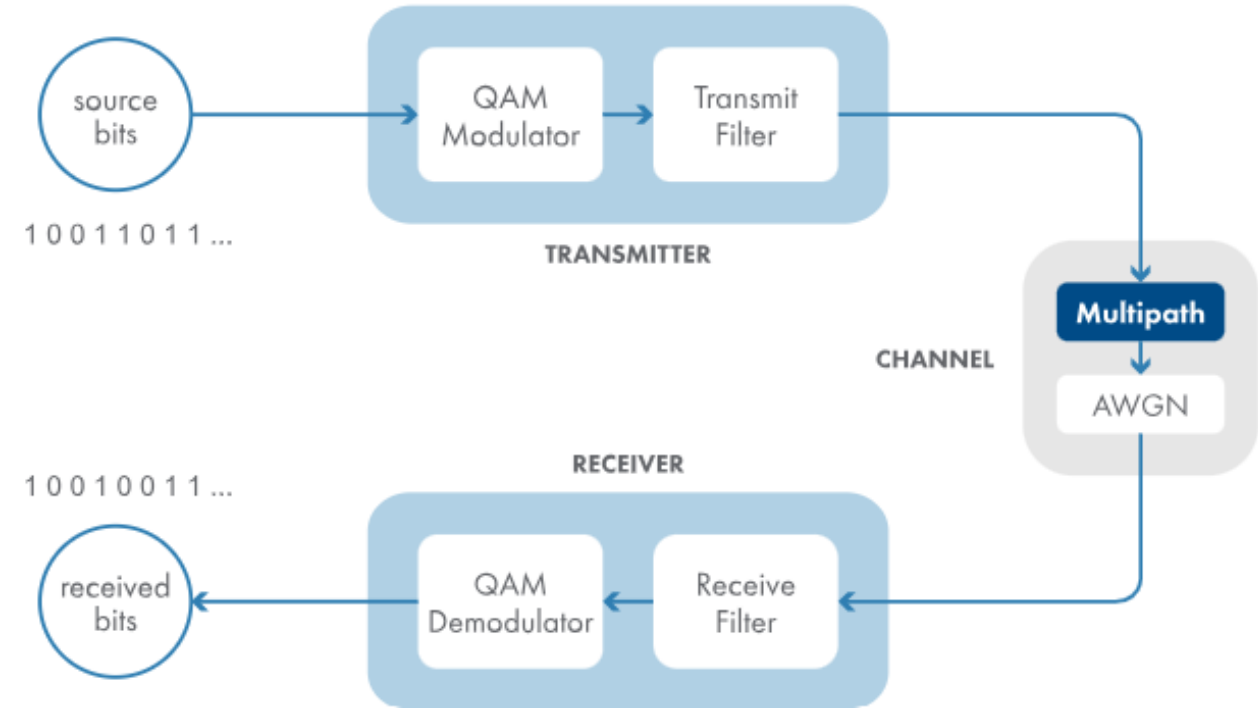
$h[n]$



Model a Multipath Channel

In this activity, you'll add a multipath channel to your simulation. The multipath channel precedes the AWGN because the multipath is encountered in the air, before the signal reaches the receiver that adds the AWGN.

For simplicity, you'll simulate a static channel. However, in scenarios where the transmitter, receiver, or reflectors are moving, the channel impulse response is time varying.



References

- [1] <https://www.tutorialsmate.com/2021/11/difference-between-data-and-information.html>
- [2] <https://physics.stackexchange.com/questions/314695/electromagnetic-wave-vs-electric-current#:~:text=Electric%20current%20is%20the%20movement,can%20propagate%20through%20a%20vacuum.>